

Übungsblatt: Erste Schritte mit ESP32

Hintergrund

Arduino, **ESP32**, Raspberry Pi Pico und ähnliche Boards sind wichtige Bausteine im Internet of Things, weil sie physische Messwerte aus der Umwelt erfassen, verarbeiten und an andere Systeme weitergeben können. Typische Beispiele sind Temperaturmessung, Smart-Home-Steuerung, Zustandsüberwachung, Umweltmonitoring oder industrielle Sensorik.

Grob funktionieren solche Boards so: Ein Mikrocontroller liest über GPIO-, I2C-, SPI- oder UART-Schnittstellen Sensoren und andere Hardware aus, verarbeitet die Daten in einem Programm und steuert auf dieser Basis Ausgaben wie LEDs, Displays, Motoren oder Netzwerkkommunikation. Besonders der ESP32 ist für IoT sehr beliebt, weil er im Gegensatz zu klassischen Arduino-Boards bereits WLAN und Bluetooth integriert und dadurch direkt mit Webdiensten oder MQTT-Brokern kommunizieren kann.

Programmiert werden diese Plattformen typischerweise in C/C++ mit der Arduino-IDE oder – bei vielen Boards wie dem ESP32 – auch in **MicroPython**. Für Lehrveranstaltungen ist das besonders praktisch, weil Studierende sowohl hardwarenahe Programmierung als auch Netzwerk- und Protokollkonzepte an realistischen, aber überschaubaren Beispielen erlernen können.

Wokwi ist eine browserbasierte Simulationsumgebung für Elektronikprojekte. Laut Wokwi-Dokumentation können dort unter anderem Arduino, ESP32, STM32 und verschiedene Sensoren simuliert werden; außerdem unterstützt Wokwi WiFi-Simulation sowie Protokolle wie MQTT, HTTP und NTP, was es besonders geeignet für erste IoT-Experimente macht

Lernziele

Nach Bearbeitung der Übung sollte Sie

- die Rolle von Mikrocontroller-Boards im IoT erklären können.
- den grundsätzlichen Aufbau eines IoT-Systems aus Sensor, Mikrocontroller, Kommunikation und Ausgabe verstehen.
- ein bestehendes Wokwi-Projekt starten, ausführen und verändern können.
- Sensordaten per MQTT an einen Broker senden und die Nachrichten dort beobachten können.
- ein bestehendes System um zusätzliche IoT-Komponenten wie OLED-Display und LED-Bargraph erweitern können

Ausgangslage

Verwenden Sie als Startpunkt das folgende Wokwi-Projekt: [MicroPython MQTT Weather Logger \(ESP32\)](https://wokwi.com/esp32) (bzw. <https://wokwi.com/esp32> → MicrypPython MQTT Weather Logger). Dieses Projekt misst Temperatur und Luftfeuchtigkeit mit einem DHT22-Sensor und sendet die Werte per MQTT an einen Broker; in der Projektbeschreibung wird außerdem erläutert, wie die gesendeten Werte im MQTT-Webclient beobachtet werden können. Optional: Wenn Sie sich mit einem (Wegwerf-)Account anmelden, können Sie ihr Projekt auch online speichern, sonst nur per Zip runterladen (ist auch gut).

Aufgabe 1: Projekt laden und ausprobieren

1. Öffnen Sie das Wokwi-Projekt im Browser.
<https://wokwi.com/esp32> → MicrypPython MQTT Weather Logger
2. Starten Sie die Simulation.
3. Verschaffen Sie sich einen Überblick über:
 - den verwendeten Mikrocontroller,
 - den DHT22-Sensor,
 - den Quellcode,
 - die serielle Ausgabe.

Arbeitsauftrag:

Beschreiben Sie in 3–5 Sätzen, welche Funktion das Ausgangsprojekt erfüllt und welche Komponenten daran beteiligt sind.

Aufgabe 2: MQTT-Nachrichten beobachten

Folgen Sie der Anleitung, die im Programmtext des Projekts beschrieben ist: Öffnen Sie den angegebenen MQTT-Webclient, verbinden Sie sich mit dem Broker und abonnieren Sie das Topic `wokwi-weather`. Wenn Sie anschließend in Wokwi die Werte des DHT22-Sensors ändern, sollten neue MQTT-Nachrichten im Client erscheinen.

Arbeitsauftrag:

- Stellen Sie die Verbindung zum MQTT-Broker her.
- Abonnieren Sie das richtige Topic.
- Verändern Sie Temperatur und Luftfeuchtigkeit im Wokwi-Sensor.
- Dokumentieren Sie anhand von mindestens drei Änderungen, wie sich die gesendeten Werte verändern.

Aufgabe 3: Erweitern Sie den Code

Die Übung geht davon aus, dass eine Übermittlung von Sensorwerten, nur bei Änderungen stattfindet. Fügen Sie für die Übertragung noch, die Zeit hinzu, so dass immer die aktuelle Uhrzeit noch mit gesendet wird in der MQTT Message an den MQTT Broker.

Arbeitsauftrag:

1. Erweitern Sie den Code, so dass in der MQTT Nachricht noch die aktuelle Zeit mitgesendet wird.
2. Evtl. müssen Sie die Zeit noch per Network Time Protocol (NTP) aktualisieren, finden Sie dazu ein Beispiel / Code Snippet auf <https://wokwi.com/esp32>

Aufgabe 4: Erweiterung um ein SSD1306-Display

Erweitern Sie das System um ein SSD1306-OLED-Display, das die aktuell gemessenen Temperatur- und Luftfeuchtwerte direkt im simulierten System anzeigt. Wokwi unterstützt ein solches 128x64-I2C-OLED; die Dokumentation nennt dafür standardmäßig die I2C-Adresse `0x3C`, und für neue Projekte soll `board-ssd1306` verwendet werden.

Arbeitsauftrag:

3. Binden Sie ein SSD1306-OLED-Display in die Schaltung ein (Siehe <https://wokwi.com/esp32>)
4. Verbinden Sie das Display über (Inter-Integrated Circuit) I2C mit dem Mikrocontroller.
5. Erweitern Sie den Code so, dass die aktuellen Messwerte auf dem Display erscheinen.

Mindestanforderung an die Anzeige:

- Zeile 1: Temperatur
- Zeile 2: Luftfeuchtigkeit
- Aktualisierung bei jeder neuen Messung

Aufgabe 5: Erweiterung um einen 10-Segment-LED-Bargraph

Ergänzen Sie zusätzlich einen `wokwi-led-bar-graph`, der die Temperatur als Skalenwert visualisiert. Der Bargraph soll niedrige Temperaturen mit wenigen aktiven Segmenten und hohe Temperaturen mit mehr aktiven Segmenten darstellen.

Arbeitsauftrag:

1. Fügen Sie einen 10-Segment-LED-Bargraph in das Projekt ein.
2. Verbinden Sie ihn mit geeigneten GPIO-Pins des Mikrocontrollers.
3. Entwickeln Sie eine Zuordnung von Temperaturwerten zu leuchtenden Segmenten, zum Beispiel von „sehr kalt“ bis „sehr warm“.
4. Testen Sie die Darstellung mit verschiedenen Temperaturwerten.

Hinweis:

Eine mögliche Lösung ist, einen Temperaturbereich – etwa von -20 °C bis 60 °C – auf 0 bis 10 aktive LEDs abzubilden.

Aufgabe 5: Freie Erweiterung bei verbleibender Zeit

Wenn Sie die Pflichtaufgaben abgeschlossen haben, erweitern Sie das Projekt um mindestens **eine weitere IoT-Komponente oder Funktion**. Wokwi stellt dafür zahlreiche weitere Bauteile und Sensoren bereit und erlaubt es, Projekte direkt im Browser zu verändern und zu testen.

Mögliche Erweiterungen:

- Grenzwertalarm bei hoher Temperatur, zusätzliche Status-LED, Summer/Buzzer bei Alarm, weiterer Sensor, anderes MQTT-Topic, Empfang von MQTT-Befehlen, ein Regler für die Messwerte zum drehen, Einbindung von Knöpfen zum Umschalten der Anzeige ...